Clase de Complexitate (1)

2023

Cuprins

1 Motivație **1**

2 Obiective **1**

3 Convenții **2**

4 Clase de Complexitate **2**
4.1 Complexitate 2
4.2 Mașina Turing Deterministă 2
4.3 Mașina Turing Nedeterministă 3
4.4 Exemplu Algoritm Nedeterminist 3

5 Clasa PTIME (P) **4**

6 Clasa NPTIME (NP) **4**

7 Clasa EXPTIME (EXP) **5**

8 P vs NP **5**

9 Cele mai grele probleme din NP **6**

1 Motivație

În cursul precedent ne-am propus să explorăm limitele calculabilității. Am descoperit astfel că există probleme pentru care nu putem implementa o soluție care să meargă pentru orice fel de intrare, indiferent de modelul de calcul ales. În continuare, ne propunem să clasificăm problemele care sunt calculabile în funcție de abilitatea noastră de a le rezolva eficient. Vom descoperi că există o categorie vastă de probleme, cu aplicații practice în foarte multe domenii, pentru care nu am descoperit soluții care să funcționeze *corect* și să consume o cantitate *rezonabilă* de resurse (ex: timp, spațiu) pentru *orice* intrare validă. Discuția din acest capitol se va concentra pe analiza dificultății problemelor de decizie[1].

2 Obiective

Ne propunem să:

- Identificăm principalele clase de complexitate.
- Explicăm conceptul de algoritm nedeterminist.
- Comparăm dificultatea mai multor probleme de referință, folosind mecanismul de reducere polinomială.

[1] În multe situații practice, putem reformula o problemă de optimizare (ex: '*Care este costul minim pentru a realiza X?*') ca o problemă de decizie ('*Putem realiza X cu un cost de cel mult Y?*'). Astfel, dacă putem rezolva eficient problema de decizie, respectiv să explorăm eficient spațiul valorilor posibile Y, am avea o soluție viabilă și pentru problema inițială de optimizare. De exemplu, putem spune că problema calculării drumului cu cele mai puține muchii intermediare între două noduri dintr-un graf orientat este *cel puțin la fel de grea* ca problema verificării dacă există un drum între A și B cu cel mult K muchii. Prin urmare, dacă știm că problema de decizie este *grea*, în sensul că nu am descoperit o soluție eficientă pentru a o rezolva, și problema corespunzătoare de optimizare va fi *grea*.

3 Convenții

Pentru a simplifica discuția, în acest curs vom folosi Mașina Turing (not. MT) cu o singură bandă ca model de calcul de referință. Pentru a reprezenta intrarea unei probleme vom presupune o codificare binară ($a = 9 \implies '1001'$). Vom nota cu n dimensiunea datelor de intrare, respectiv numărul de biți necesari codificării datelor de intrare.

4 Clase de Complexitate

4.1 Complexitate

Definim funcțiile de complexitate $time_M : \mathbb{N} \to \mathbb{N}$, respectiv $space_M : \mathbb{N} \to \mathbb{N}$, particularizate pentru o Mașină Turing M, care se oprește pentru orice intrare. Atunci:

- $time_M(n)$ (complexitatea temporală) descrie numărul maxim de pași realizați de mașina M pentru orice intrare de dimensiunea n.
- $space_M(n)$ (complexitatea spațială) descrie numărul maxim de celule scanate de M pentru orice intrare de dimensiunea n.[2]

Ne propunem să grupăm problemele în funcție de complexitatea celei mai eficiente MT care le rezolvă.

Definiție 1: Clase de complexitate

$TIME(t(n))$ = mulțimea problemelor de decizie rezolvate de o MT într-un număr de pași $time_M(n) \in O(t(n))$.
$SPACE(s(n))$ = mulțimea problemelor de decizie rezolvate de o MT cu complexitate spațială $space_M(n) \in O(s(n))$.

4.2 Mașina Turing Deterministă

Pentru a defini o MT deterministă avem nevoie de:

- Q - mulțimea stărilor. În particular, notăm:
  - q_{init} - Starea inițială.
  - q_{accept} - Starea finală, corespunzătoare acceptării intrării.
  - q_{reject} - Starea finală, corespunzătoare respingerii intrării.
- Σ - alfabetul, respectiv mulțimea valorilor care pot fi stocate într-o celulă. Vom include în alfabet un simbol default (blank) care semnifică faptul că pe celula respectivă nu a fost încă scrisă o valoare validă.
- $\delta : Q \times \Sigma \to Q \times \Sigma \times \{L, R\}$[3] - funcția de tranziție
  - De exemplu, dacă $\delta(q_1, 0) = (q_2, 1, L)$ atunci, dacă MT se află în starea q_1, iar la poziția curentă a capului de control se află scrisă valoarea '0', MT va trece în starea q_2, va scrie valoarea '1' și va muta capul de bandă un pas la stânga.
  - Prin convenție, capul de bandă nu se va mișca la stânga primei poziții de pe bandă. În schimb, se poate deplasa arbitrar de mult în dreapta.

Prin urmare, putem descrie complet **configurația** unei MT de la un moment de timp prin starea în care se află MT, valorile scrise pe bandă (T) și poziția capului. Redefinim conceptul de clasă de complexitate pentru a ține cont de faptul că folosim o MT *deterministă*.

[2] În practică, atunci când ne referim la complexitatea spațială asimptotică a unui algoritm, ne propunem să măsurăm doar memoria suplimentară celei necesare pentru a reprezenta datele de intrare. În acest mod putem diferenția între algoritmi care folosesc spațiu adițional subliniar (e.g. $O(log(n))$ vs. $O(\sqrt{n})$).

[3] Valoarea 'L' corespunde unei mutări ale capului de bandă la stânga cu o poziție, respectiv 'R' corespunde unei mutări la dreapta.

Definiție 2: Clase de complexitate (2)

$DTIME(t(n))$ = mulțimea problemelor rezolvate de o MT *deterministă* cu o complexitate temporală $time_M(n) \in O(t(n))$.
$DSPACE(s(n))$ = mulțimea problemelor rezolvate de o MT *deterministă* cu o complexitate spațială $space_M(n) \in O(s(n))$.

4.3 Mașina Turing Nedeterministă

Observăm că, dintr-o configurație validă a MT **o singură** tranziție este posibilă. Dacă într-un punct din procesul de rezolvare avem K alegeri posibile (unde K este un număr cunoscut, finit), va trebui să le investigăm pe rând. Dar dacă MT are putea analiza simultan consecințele tuturor alegerilor? (Oare am putea rezolva *eficient* mai multe probleme?)

Definim un model de calcul teoretic care ne oferă această posibilitate. Astfel, un model de calcul nedeterminist, precum Mașina Turing nedeterministă (MTN), permite, în plus față de un model determinist, să realizăm o acțiune specială $choice(S)$. $choice(S)$ primește la intrare o mulțime finită de valori și construiește un fir de execuție pentru fiecare valoare posibilă din mulțimea S. Acestea se vor executa simultan până ce modelul de calcul ajunge într-o stare finală corespunzătoare răspunsului DA, pentru cel puțin unul din firele de execuție, sau pentru toate firele ajunge la răspunsul NU. Prin convenție, considerăm că $choice(S)$ se execută în timp constant.

O perspectivă alternativă ar fi următoarea: Atunci când apelăm $choice(S)$, modelul *ghicește* care dintre opțiuni ne permite să oferim răspunsul DA (dacă acest lucru este posibil) și va ghida procesul de calcul în aceea direcție, ignorând celelalte opțiuni din S.

Complexitatea totală a unui algoritm nedeterminist este dată de suma complexităților operațiilor din secvența cea mai scurtă care termină algoritmul cu succes[4]. Vom folosi următoarea notație pentru a grupa problemele în funcție de abilitatea noastră de a le rezolva folosind un algoritm nedeterminist:

Definiție 3: Clase de complexitate (3)

$NTIME(t(n))$ = mulțimea problemelor rezolvate de o MT *nedeterministă* cu o complexitate temporală $time_M(n) \in O(t(n))$.
$NSPACE(s(n))$ = mulțimea problemelor rezolvate de o MT *nedeterministă* cu o complexitate spațială $space_M(n) \in O(s(n))$.

4.4 Exemplu Algoritm Nedeterminist

Problema 1: K-Colorare

Date de intrare: Graf $G(V, E)$, $K \in \mathbb{N}_{>0}$
Date de ieșire: Putem atribui o culoare c ($1 \leq c \leq K$) fiecărui nod din graf, astfel încât fiecare nod să fie colorat diferit față de vecinii săi?
Certificat[a]: O secvență de $|V|$ valori, corespunzătoare culorilor atribuite fiecărui nod din graf.

[a] Numim certificat o secvență de caractere pe care o putem folosi pentru a demonstra că o problemă de decizie are soluția "DA" pentru un set de date de intrare w.

Acestă problemă de decizie poate fi rezolvată nedeterminist în felul următor: Mai întâi, construim certificatul. Astfel, vom alege pe rând, pentru fiecare nod, o culoare din mulțimea $\{1, \ldots, K\}$, folosind *choice*. Apoi, verificăm dacă alegerile făcute sunt corecte, analizând dacă fiecare muchie din graf leagă doar noduri de culori diferite.

Prin convenție, în pseudo-cod marcăm cu comanda $fail$ faptul că algoritmul se termină cu insucces, respectiv cu *success* dacă am găsit un certificat valid.

[4] Astfel definită, complexitatea mai poartă numele de complexitate angelică [1]

```
function K-Colour(G(V, E), K)
    colourMap = Map()
    for node in V do
        colour = choice({1..K})
        colourMap[node] = colour
    end for
    for edge(u, v) in E do
        if colourMap(u) == colourMap(v) then
            fail.
        end if
    end for
    success.
end function
```

Deoarece *choice*() se realizează în $O(1)$, complexitatea temporală a întregului algoritm este: $\theta(|V|) + \theta(E)$. Complexitatea spațială este $O(|V|)$, deoarece reținem cele $|V|$ alegeri în structura $colourMap$.

Dacă ne-am propune să simulăm acest algoritm pe o mașină de calcul deterministă, construirea certificatului s-ar realiza în $K^{|V|}$ (deoarece acesta este numărul total de moduri în care putem colora cele $|V|$ noduri folosind K culori)[5]. Din păcate, pentru acesta problemă nu s-a descoperit încă o soluție deterministă cu o complexitate temporală subexponențială.

5 Clasa PTIME (P)

Definiție 1: Clasa PTIME (sau P)

Mulțimea problemelor de *decizie* care pot fi rezolvate de un algoritm determinist **polinomial**.

$$P = \bigcup_k DTIME(n^k)$$

Exemple:

- Există un element X într-o listă L?
- Există un drum elementar de la nodul A la nodul B într-un graf orientat?
- Este un graf bipartit?

Considerăm că în clasa P se află, *în general*, acele probleme care pot fi rezolvate folosind o cantitate *rezonabilă* de resurse[6]. Bineînțeles, aceasta este o simplificare a realității (de exemplu, ignorăm faptul că valoarea lui k poate să fie arbitrar de mare), dar așa cum vom vedea, este o simplificare utilă din punct de vedere al comparării dificultății problemelor. Este puțin probabil să întâlnim un algoritm practic de complexitate n^{100}. Există în schimb multe probleme importante pentru care nu s-au descoperit decât soluții suprapolinomiale.

6 Clasa NPTIME (NP)

Definiție 2: NPTIME (sau NP)

Mulțimea problemelor de *decizie* care pot fi rezolvate de un algoritm *nedeterminist* **polinomial**.

$$NP = \bigcup_k NTIME(n^k)$$

[5] Mai general, dacă dimensiunea certificatului de care avem nevoie este de N simboluri, atunci pentru a-l construi sunt suficiente N apelări ale funcției *choice*(S). Simularea deterministă a construirii sale s-ar realiza în $|S|^N$.

[6] Problemele din P mai apar în literatură sub denumirea de probleme *tractabile* [1].

Definiție 3: NPTIME (sau NP)

Alternativ: Mulțimea problemelor de decizie care admit existența unui *certificat* c care poate fi verificat în timp *determinist* polinomial. (implicit, $|c| \leq p(n)$, unde $p()$ este un polinom)

Exemple:

- Există un drum elementar cu cel *puțin* K noduri intermediare de la A la B?
- Putem planifica n procese, cu constrângeri de timp (deadline, durată), pe m mașini?
- K-Colorare.

Dacă P descrie clasa problemelor de decizie pentru care putem **construi eficient**[7] un certificat, în clasa NP sunt grupate acele probleme pentru care putem să **verificăm eficient** dacă un certificat de dimensiune polinomială demonstrează că problema are soluția DA.

7 Clasa EXPTIME (EXP)

Definiție 4: Clase de complexitate(4)

$$EXPTIME \text{ (sau EXP)} = \bigcup_k DTIME(2^{n^k})$$

Exemple:

- Se oprește o MTD în K pași pe o intrare w?
  - Deși putem afla răspunsul în K pași (simulând MTD pe intrarea w), complexitatea este exponențială relativ la numărul de biți necesar pentru a codifica datele de intrare ($log_2(w)$, conform convenției asumate).
- Verificarea dacă o mutare garantează victoria într-un joc generalizat de șah/Go, pe o tablă cu $n \times n$ celule.
  - Mai general, o problemă de decizie nu se poate afla în PTIME dacă avem nevoie de un certificat cu o dimensiune suprapolinomială pentru a valida dacă problema are soluția DA pe o anumită intrare. De exemplu, la șah, se pot realiza un număr exponțial de mutări de la starea inițială până la o stare finală.

8 P vs NP

Sunt cele două clase de probleme diferite ($P \neq NP$)? Din păcate, nu știm răspunsul la această întrebare, deși ne-am dori foarte mult să îl aflăm. Astfel, dacă cele două clase ar fi echivalente, atunci multe probleme cu aplicații practice importante ar admite, teoretic, o rezolvare eficientă.

Știm însă că:

- $P \subseteq NP$
  - Dacă $Q \in P$, atunci există un algoritm determinist polinomial A care rezolvă Q. Algoritmul poate rula, nemodificat, în timp polinomial, pe un model de calcul nedeterminist.
- $NP \subseteq EXP$
  - Dacă $Q \in NP$, atunci există un algoritm *nedeterminist* A care rezolvă Q în timp polinomial ($p(n)$). Fie b, numărul maxim de tranziții posibile la fiecare pas. Atunci, întregul arbore de execuție are cel mult $b^{p(n)}$ noduri. Astfel, putem simula determinist toate deciziile luate de algoritmul A în timp $2^{O(p(n))}$.
  - (Alternativ) Dacă $Q \in NP$, putem verifica, în timp determinist polinomial, dacă un certificat c ($|c| \leq p(n)$) demonstrează că problema are soluția "DA" pentru o anumită intrare. Atunci, putem rezolva problema generând toate string-urile posibile de dimensiune $p(n)$ (în timp $b^{p(n)}$) și aplicând algoritmul de verificare pentru fiecare.

[7] În acest context, folosim termenul de eficient ca sinonim pentru timp polinomial determinist.

9 Cele mai grele probleme din NP

Pentru a descoperi *cele mai grele* probleme din clasa NP introducem mecanismul de **reducere**.

Definiție 4: Reducere polinomială (many-one)

Fie A și B, două probleme de decizie. Fie o funcție $F : Input_A \rightarrow Input_B$, definită pe intrările corespunzătoare problemei A, care produce intrări valide pentru problema B. Prespunem că F poate fi calculată de un algoritm determinist cu complexitate temporală polinomială. Atunci, spunem că A se reduce la B (not: $A <_p B$) dacă:

$$A(w) = 1 \iff B(F(w)) = 1$$

Dacă $A <_p B$, atunci putem rezolva A, apelând direct algoritmul corespunzător lui B pe intrarea modificată. Astfel, putem să facem câteva afirmații referitoare la dificultatea relativă a rezolvării celor două probleme:

- Complexitatea rezolvării problemei B ne oferă o limită *superioară* pentru oricare rezolvare directă a problemei A.
- Complexitatea rezolvării problemei A ne oferă o limită *inferioară* pentru oricare rezolvare directă a problemei B.

Observăm că relația este tranzitivă. Astfel, $A <_p B$ și $B <_p C$ implică $A <_p C$.

Definiție 5: NP-Hard

A $\in$ NP-Hard $\iff \forall Q \in NP, Q <_p A$

În NP-Hard se află toate problemele *cel puțin* la fel de grele ca toate problemele din NP. În NP-Hard se află însă și probleme mai grele decât cele din NP (de exemplu, Halt, problema terminării programelor).

Definiție 6: NP-Complete

A $\in$ NP-Complete $\iff A \in$ NP-Hard AND $A \in NP$.

Soluția unei probleme NP-Complete (NPC) poate fi folosită pentru a rezolva **orice** problemă din NP. Din acest motiv, considerăm, informal, că problemele NPC sunt *cele mai grele* probleme din NP.

În cursul următor vom descoperi că:

- Există cel puțin o problemă NP-Completă (Teorema Cook) - Problema SAT.
- SAT se reduce direct (sau indirect) la un număr vast de probleme practice din NP. Prin urmare, toate aceste probleme sunt și ele în NPC.

Referințe

[1] Cristian A Giumale. *Introducere în analiza algoritmilor: teorie şi aplicaţie*. Polirom, 2004.

[2] Charles E Leiserson et al. *Introduction to algorithms*. MIT press, 2009.

[3] Tim Roughgarden. *Algorithms illuminated*. Soundlikeyourself publishing, 2017.

[4] Michael Sipser. *Introduction to the Theory of Computation*. English. 3rd ed. Cengage Learning, 2012, p. 504. ISBN: 978-1133187790.